Data Structures and Algorithm

COURSE CODE: CS-201

TOPIC: ARRAYS

COURSE INSTRUCTOR: HABIBA ARSHAD

CLO Covered

CLO2: **Apply** and design various data structures methods for specified applications.

Inserting and Deleting an Element

Insertion: Adding an element

- Beginning
- Middle
- End

Deletion: Removing an element

- Beginning
- Middle
- End

Insertion in Array

- Inserting an element at the “end” of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element.

- On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must be moved downward to new locations to accommodate the new element and keep the order of the other elements.

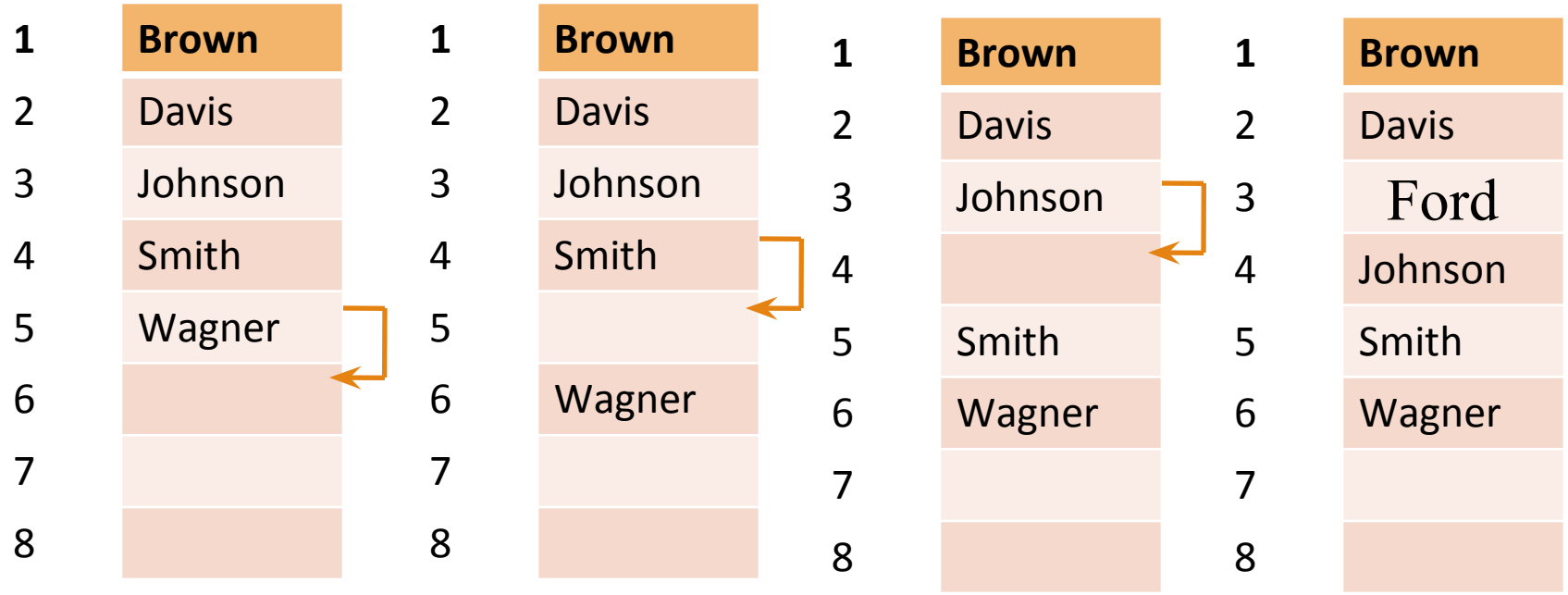
Insertion

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	Wagner
6	
7	
8	

1	**Brown**
2	Davis
3	Johnson
4	Smith
5	Wagner
6	Ford
7	
8	

Insert Ford at the End of Array

Insertion



Insert Ford as the 3rd Element of Array

Insertion is not Possible without loss of data if the array is FULL

Insertion Algorithm

Algorithm inserts a data element ITEM into the Kth position in a linear array LA with N elements i.e. INSERT(LA,N,K,ITEM).

Here LA is a linear array with N elements and K is a positive integer such that K<=N. This algorithm inserts an element ITEM into the Kth position in LA.

1. [Initialize counter] Set J := N
2. Repeat Steps 3 and 4 while J ⩾ K.
3. [Move Jth element downward] Set LA[J+1] := LA[J]
4. [Decrease counter] Set J : = J-1
   [End of Step 2 loop]
5. [Insert element] Set LA[K] := ITEM
6. [Reset N] Set n:= N+1
7. Exit

Time Complexity

The running time for inserting an element in an array depends on where you are inserting it:

1. **At the End**: If you're inserting at the end of the array, the time complexity is O(1), because you simply add the element without shifting any other elements.
2. **At the Beginning or Middle**: If you're inserting at the beginning o r any position in the middle, the time complexity is O(n), where n i s the number of elements in the array. This is because you need to shif t all subsequent elements one position to the right to make space for t he new element.

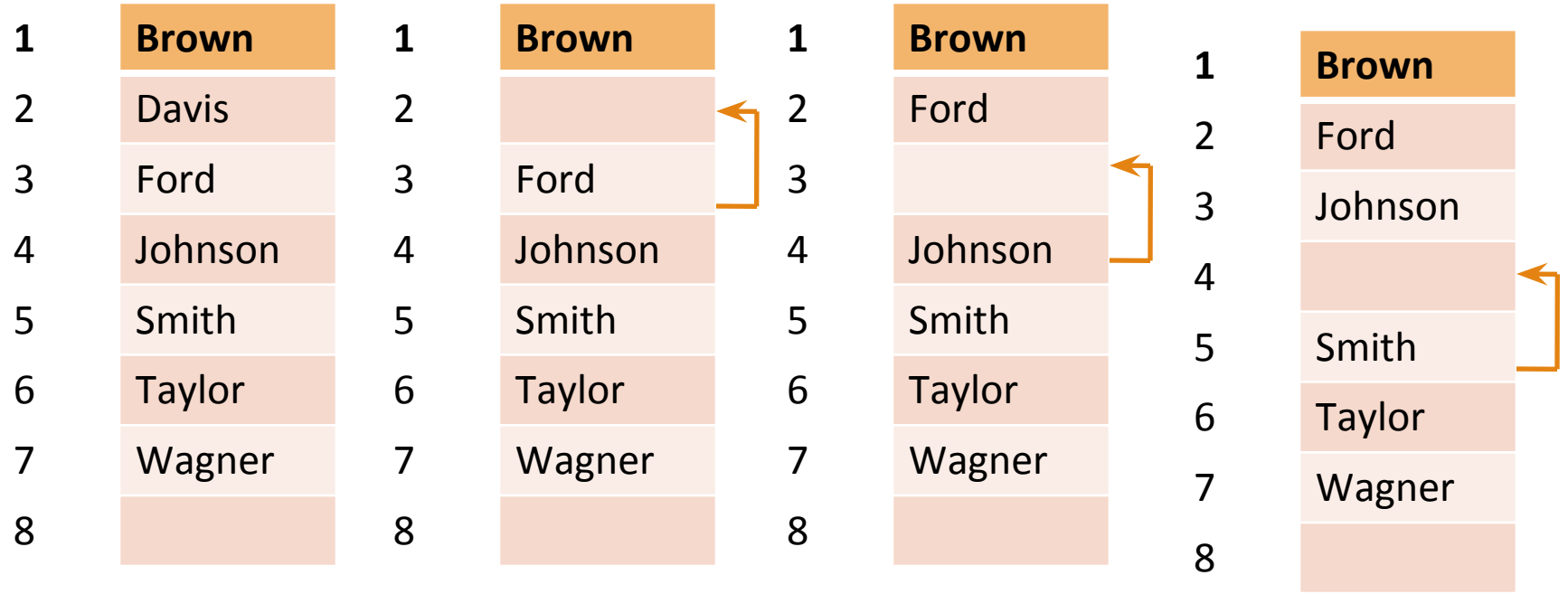
Deletion

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	Wagner
8	

1	**Brown**
2	Davis
3	Ford
4	Johnson
5	Smith
6	Taylor
7	
8	

Deletion of Wagner at the End of Array

Deletion



Deletion of Davis from the Array

Deletion

1	**Brown**
2	Ford
3	Johnson
4	Smith
5	Taylor
6	Wagner
7	
8	

No data item can be deleted from an empty array

Algorithm for deletion-Steps

Underflow

Find location of element

Reorganize the whole array

Deletion Algorithm

The following algorithm deletes the Kth element from a linear array LA and assigns it to a variable ITEM i.e. DELETE(LA,N,K,ITEM).

Here LA is a linear array with N elements and K is a positive integer such that K<=N. This algorithm deletes the Kth element from LA.

```
1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set LA[J] = LA[J + 1]
5. Set J = J+1
6. Set N = N-1
7. Stop
```

Time Complexity

The running time for deleting an element in an array, like insertion, depends on its position:

From the End: Deleting from the end is O(1) because you simply remove the last element without needing to shift any other elements.

From the Beginning or Middle: Deleting from the beginning or any position in the middle is O(n), where n is the number of elements in the array. This is because you need to shift all subsequent elements one position to the left to fill the gap left by the removed element.

Questions?